

Dynamo + KV Cache

SC 26 | Taming AI Storage Systems

NVIDIA Dynamo

Data center scale open-source inference-serving framework.

An orchestration layer above the inference engines.

- **Inference Engine Agnostic**: vLLM, SGLang and TensorRT-LLM.
- **Disaggregated Serving**: Separates the context-processing ("prefill") phase from the token-generation ("decode") to specialized worker pools.
- **KV-Aware Routing**: Routes prompts to GPUs that already have the matching KV cache values.
- **Multi-Tier KV Cache Management**: transfers KV cache memory dynamically across GPU HBM, CPU RAM and storage tiers.

KV-Cache

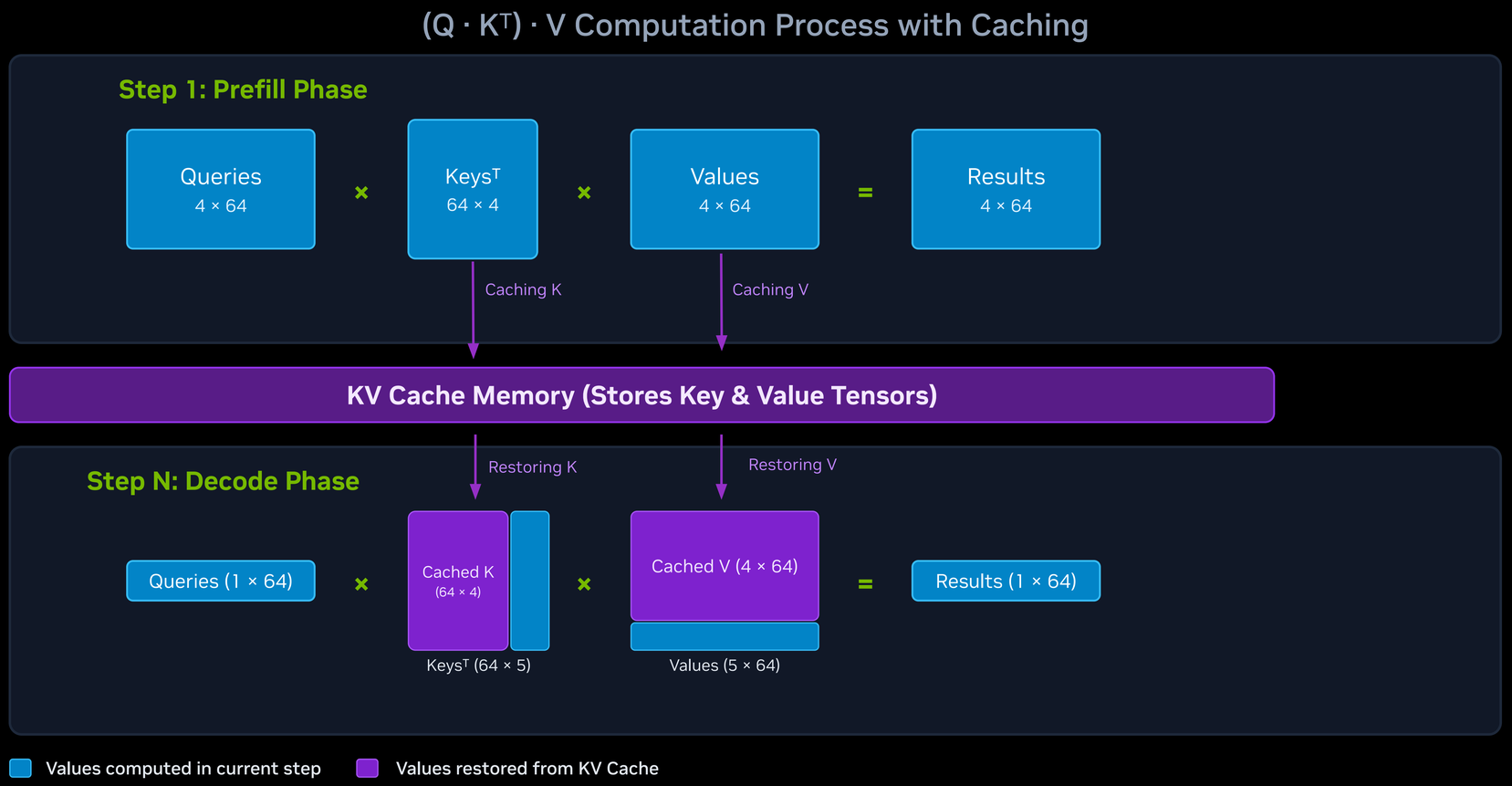
A Key-Value Cache “KV$” is an optimization technique that caches attention computations.

This avoids the need for recomputing these from scratch at every step, and for repeated contexts.

KV$ Reduces answer generation time and the amount of computed tokens.

NVIDIA

KV-Cache

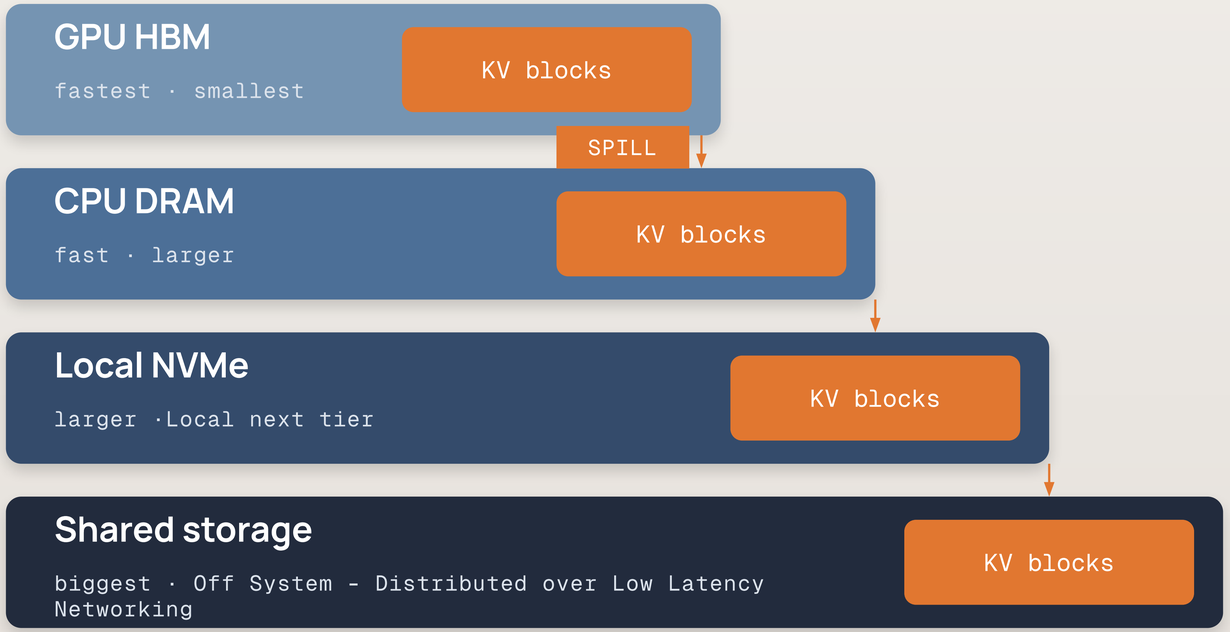


- **Query (Q)**: A vector asking, "What am I looking for?"
- **Key (K)**: A vector offering, "What do I offer?"
- **Value (V)**: A vector providing, "What is my actual content?"

NVIDIA

What KV cache is — and why it spills

Training made storage a bandwidth problem. Inference makes it a latency + capacity problem.



THE MECHANISM

1. The KV cache holds every token's attention keys/values so the model doesn't recompute them.
2. It grows with context length × concurrent requests — and quickly outgrows GPU HBM.
3. So it spills down the tiers: HBM → DRAM → local NVMe → shared storage.
4. **Now a storage read moves TTFT (a cache hit skips prefill) and ITL (decode-step reads).**

NVMe Non-Volatile Memory Express **TTFT** Time To First Token **ITL** Inter-Token Latency **KV** Key-Value **HBM** High Bandwidth Memory **DRAM** Dynamic Random-Access Memory

KV CACHE · ZOOM OUT

Where storage plugs into the serving stack

Public API / gateway	auth · rate limiting · tenant routing
Resource management	Kubernetes · GPU/CPU provisioning
Model selection & orchestration	router · policy · cache-aware placement
Distributed serving + KV / prompt caching	replicas · KV manager (LMCache, Dynamo)
Core inference	vLLM · TensorRT-LLM · SGLang · PagedAttention
Model layer	weights · adapters · versions

Tenancy & QoS on the data path
per-tenant isolation and fair-share where many tenants share one tier

Retrieval corpus
documents, chunks, embedding artifacts beside the serving fleet; the vector index stays with the retrieval service

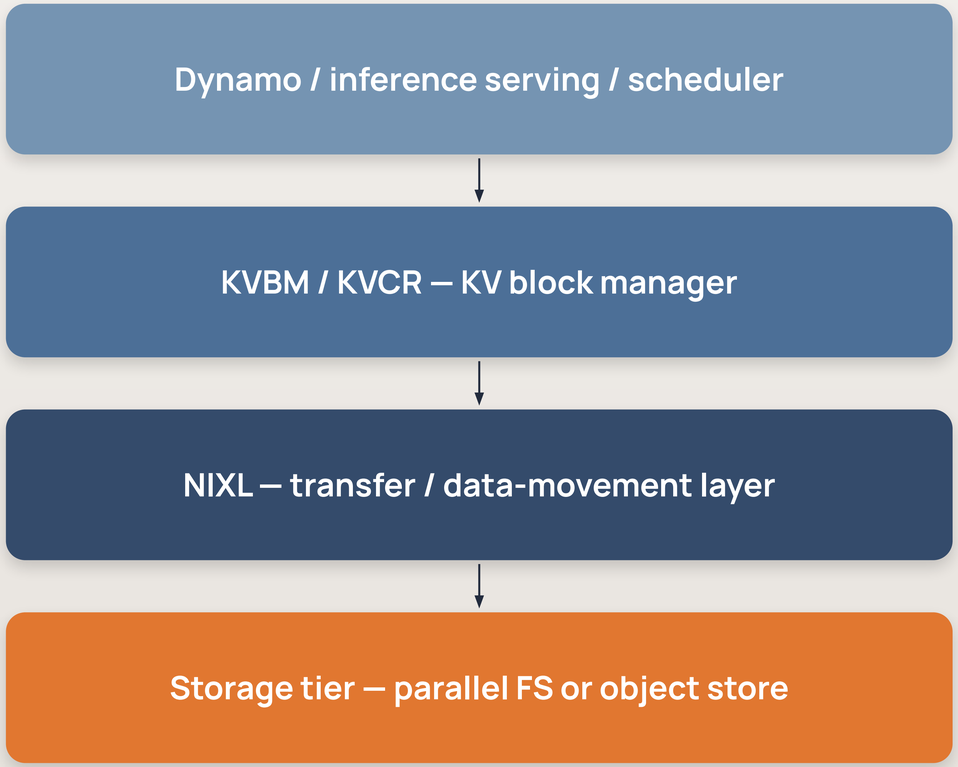
Shared KV / prefix tier
KV manager → NIXL → shared tier; compatible prefixes reused across workers, not pinned to one GPU

Model store
weights, adapters, versions as shared objects; the loader/streamer pulls them — cold-start → TTFT

Layers after Hands-On LLM Serving and Optimization, Ch. 4 (redrawn). Shared tier = parallel FS or object/KV store — several vendors, DDN among them.

TTFT Time To First Token **KV** Key-Value **NIXL** NVIDIA Inference Xfer Library **QoS** Quality of Service **LLM** Large Language Model **FS** File System

How the offload actually happens



KVCR is planned to succeed KVBM

ABOVE NIXL

The serving-stack internals: Dynamo itself, BlueField SNAP, DOCA, and the plugin implementations built around the KV block manager.

BELOW NIXL

The storage tier NIXL lands on — how the same striping, tiering and metadata behavior from the overview shows up as KV-cache read latency.

The same stack shape serves both backend types — file-system backends (GPUDirect Storage / POSIX io_uring) and KV-native backends (CUDA-direct). Dynamo, vLLM, SGLang and LMCache all ride it.

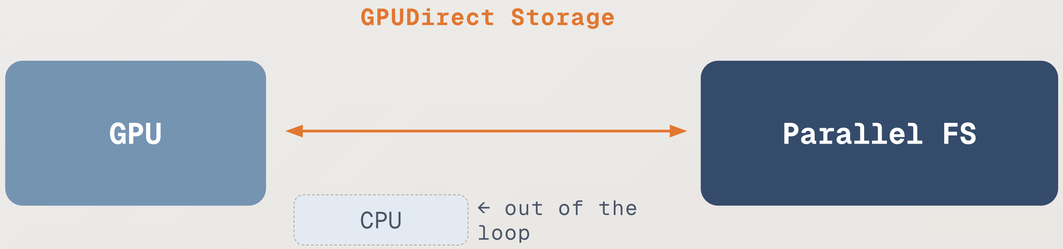
KV Key-Value **NIXL** NVIDIA Inference Xfer Library **KVBM** KV Block Manager **KVCR** planned successor to KVBM **SNAP** Software-defined Network Accelerated Processing **DOCA** Data-Center-Infrastructure-On-A-Chip Architecture **FS** File System

KV CACHE · WHERE STORAGE PLUGS IN

Two ways a storage tier plugs into NIXL

PARALLEL-FS BACKEND

Parallel file system (e.g., Lustre / EXAScaler)



Runs with the standard Dynamo container. A GPUDirect Storage path moves KV blocks GPU↔filesystem by DMA, keeping the CPU out of the loop. Works across Dynamo, vLLM, SGLang, TensorRT-LLM.

OBJECT / KV-NATIVE BACKEND

Object / KV-native store (e.g., Infinia, S3-class stores)



KV blocks move GPU/CPU↔store by DMA + RDMA for sub-millisecond lookups. A key-value store at its core — the natural shape for KV cache. Works across Dynamo, vLLM, SGLang, LMCache.

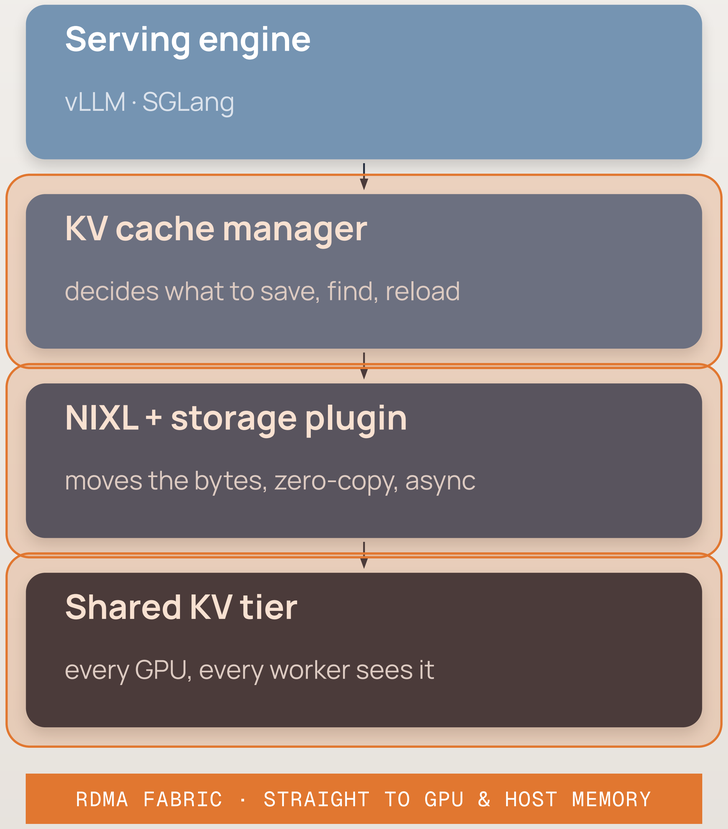
The bridge to the lab: the knobs that make KV-cache reads fast are the same striping / DoM / metadata knobs from Lab 2. Several vendors — DDN among them — are wiring both patterns into NIXL.

No benchmark numbers here by design — this session is about the architecture, not a bake-off.

DoM Data-on-MDT **S3** Simple Storage Service **RDMA** Remote Direct Memory Access **DMA** Direct Memory Access **KV** Key-Value **NIXL** NVIDIA Inference Xfer Library **LLM** Large Language Model **FS** File System

KV CACHE · WORKED EXAMPLE

One request's journey through a shared KV tier



1 **Offload** After prefill, KV blocks go from GPU/host memory to the shared tier — batched, async, off the critical path.

2 **Look up** Next request: the KV manager hunts the longest matching prefix — GPU, then host, then local NVMe, then the shared tier.

3 **Reload** Matching blocks stream back over RDMA into GPU memory. Only the new tokens get prefilled.

4 **Share** Any GPU, any worker, after a restart: one tier the whole cluster sees. The router sends the request to where the hit is.

5 **Recover** Lose a drive and the plugin reports an error; the GPU recomputes those tokens and serving continues. It's a cache.

Land step 4: the HBM prefix cache only helps the GPU that built it. A shared tier is the one every GPU sees — and it survives a restart.

Worked example: DDN's open-source Infinia plugin for NIXL (upstreamed, source readable) — the same five steps apply to any tier behind a NIXL plugin.

NVMe Non-Volatile Memory Express **RDMA** Remote Direct Memory Access **KV** Key-Value **HBM** High Bandwidth Memory **NIXL** NVIDIA Inference Xfer Library

NVIDIA DOCA Memos

A KV-Cache implementation using

- BlueField DPUs
- Spectrum-X switch
- NVMe + KV command set
- CMX - Context Memory eXtension
- DOCA software stack

